

Configure Multisite URLs

Storefront Next supports serving multiple B2C Commerce sites and locales from a single deployment. This configuration applies whether you operate a single site or multiple sites—in both cases, the multisite middleware handles site and locale resolution. Configure how site and locale identifiers appear in your URLs—in the path, as query parameters, or a combination of both. For single-site storefronts, you can omit identifiers from the URL entirely and let the middleware resolve everything from cookies and headers.

Contents

[How Multisite URLs Work](#)

[Define Your Sites](#)[Configure URL Patterns](#)[Use Alias Maps for Clean URLs](#)[Configure Site and Locale Detection](#)[Configure Site-Context Cookies](#)[URL Pattern Use Cases](#)[Access Site Context in Components](#)[Important Considerations](#)[See Also](#)

How Multisite URLs Work

Multisite configuration controls how site and locale identifiers appear in your storefront URLs. The `url` config determines the URL pattern, and the `detection` config determines how the middleware reads site and locale back from incoming requests.

Configuration is in `config.server.ts` under three areas:

- `commerce.sites` defines the available sites and their supported locales.
- `defaultSiteId` determines the fallback site when no site can be resolved from the URL, cookie, or header.
- `url` defines the URL pattern, which includes the path prefix, query parameters, and excluded routes.

Default Template Behavior

The Storefront Next template defaults to a convention where subpage URLs include site and locale identifiers (for example, `/global/en-GB/product/123`), while the homepage (`/`) resolves site and locale from cookies instead of the URL. This makes subpage URLs shareable and deterministic, while the homepage adapts to each visitor's preferences.



		prefix or query params
Site resolution	From site_id cookie (by default)	From URL path or query params
Shareable?	Depends on implementation	Yes (deterministic from URL)

This homepage behavior is an implementation choice in the template, not a framework constraint. The `Link` component (`src/components/link/index.tsx`) and `buildUrlFromContext` helper (`src/lib/url.server.ts`) skip prefixing for `/` by default. You can modify these files to apply URL prefixes to the homepage if your storefront requires it.

Define Your Sites

Sites and locales are defined under `commerce.sites` in `config.server.ts`.

```
// config.server.ts
export default defineConfig({
  app: {
    defaultSiteId: "RefArchGlobal",
    commerce: {
      sites: [
        {
          id: "RefArchGlobal",
          defaultLocale: "en-GB",
          defaultCurrency: "GBP",
          supportedLocales: [
            { id: "en-GB", preferredCurrency: "GBP" },
            { id: "fr-FR", preferredCurrency: "EUR" },
            { id: "it-IT", preferredCurrency: "EUR" },
            { id: "ja-JP", preferredCurrency: "JPY" },
            { id: "zh-CN", preferredCurrency: "CNY" },
          ],
          supportedCurrencies: ["EUR", "GBP", "JPY", "USD"],
        },
        {
          id: "RefArch",
          defaultLocale: "en-US",
          defaultCurrency: "USD",
          supportedLocales: [{ id: "en-US", preferredCurrency: "USD" }],
          supportedCurrencies: ["USD"],
        },
      ],
    },
  },
});
```

Each field serves a specific purpose:



locale can be resolved from the URL, cookie, or header.

- **defaultCurrency** – The fallback currency when no currency preference exists.
- **supportedLocales** – An array of locales available for this site. Each entry includes an `id` and a `preferredCurrency` that is applied when the user selects that locale.
- **supportedCurrencies** – The currencies available for manual selection on this site.
- **defaultSiteId** – Defined at the `app` level, this determines which site is used on a first visit when no `site_id` cookie exists.

Note

Keep the `i18n.supportedLngs` array in sync with the locale `id` values across all entries in `commerce.sites[].supportedLocales`. A mismatch causes the locale switcher to show unsupported locales or hide valid ones.

Configure URL Patterns

The `url` config in `config.server.ts` controls how multisite URLs are constructed.

```
1 url: {
2   prefix: '/:siteId/:localeId',
3   search: '?lng=:localeId',
4   excludeRoutes: ['/resource/**', '/act:
5 }
```

- **prefix** – Path segments prepended to subpage URLs. Use `:siteId` and `:localeId` as placeholders that are replaced with the resolved values at URL build time.
- **search** – Query parameters appended to subpage URLs. Use the same `:siteId` and `:localeId` placeholder syntax.
- **excludeRoutes** – Glob patterns for routes that skip prefixing (e.g., API resource routes and server actions).

Both `prefix` and `search` are optional. Use either, both, or neither depending on your URL strategy.

Warning

`url.prefix` and `url.excludeRoutes` are protected configuration paths. They



determines the React Router route structure, which is baked into the build.

Use Alias Maps for Clean URLs

Map B2C Commerce site IDs and locale IDs to shorter, URL-friendly aliases. Define these at the `app` level in `config.server.ts`.

```
1 // config.server.ts
2 export default defineConfig({
3   app: {
4     siteAliasMap: {
5       RefArchGlobal: "global",
6       RefArch: "us",
7     },
8     localeAliasMap: {
9       "en-US": "us",
10      "en-GB": "gb",
11    },
12   },
13 });
```

With `siteAliasMap`, the site `RefArchGlobal` appears as `global` in URLs: `/global/en-GB/product/123`

With `localeAliasMap`, the locale `en-GB` appears as `gb`: `/global/gb/product/123`

Both alias maps are optional. Without them, the raw B2C Commerce IDs appear in URLs.

Configure Site and Locale Detection

The URL config (`prefix`, `search`) controls how URLs are **built**. The detection config controls how site and locale are **read back** from incoming requests. These must stay in sync.

The default detection config is:

```
1 // Default site detection
2 siteDetectionConfig: {
3   order: ['path', 'querystring', 'cookie'],
4   lookupFromPathIndex: 0,
5   lookupQuerystring: 'site',
6   lookupCookie: 'site_id',
7   lookupHeader: 'X-Site-Id',
8   caches: ['cookie'],
```



```
14   lookupFromPathIndex: 1,  
15   lookupQuerystring: 'lng',  
16   lookupCookie: 'lng',  
17   lookupHeader: 'Accept-Language',  
18   caches: ['cookie'],  
19 }
```

The middleware checks each source in the specified `order` and uses the first match.

Each field controls a specific detection behavior:

- **order** –The priority sequence for resolving the value. The middleware tries each source in order and uses the first match.
- **LookupFromPathIndex** –Which URL path segment to read when `'path'` is in the `order`. `0` means the first segment, `1` means the second.
- **lookupQuerystring** –The query parameter name to check when `'querystring'` is in the `order`.
- **lookupCookie** –The cookie name to check when `'cookie'` is in the `order`.
- **lookupHeader** –The HTTP header to check when `'header'` is in the `order`.
- **caches** –Where to persist the resolved value. `['cookie']` means the resolved value is stored in a cookie for subsequent requests.



Important

The query param key for locale must be `lng` to match the i18next cookie key and default detection config. The query param key for site defaults to `site`. If you use different keys in your `url.search` config, update the corresponding `lookupQuerystring` value in the detection config.

Configure Site-Context Cookies

The `siteContext` config in `config.server.ts` controls how site-context cookies are stored. Site-context cookies are site, locale, and currency cookies. The `siteContext` fields are optional.



```
1 // config.server.ts  
2 export default defineConfig({  
3   app: {  
4     siteContext: {
```



```
10     maxAge: 604800, // 7 days
11   },
12 },
13 },
14 });
```

- **currencyCookieName** –The cookie name used to persist the shopper's selected currency. Defaults to 'currency'.
- **cookieOptions** –Cookie attributes that are applied to all three site-context cookies: site, locale, and currency. Valid options include `httpOnly`, `maxAge`, `secure`, `sameSite`, `path`, `domain`, and so on. These options follow the standard [Cookie](#) options from React Router. Defaults to `{ path: '/', sameSite: 'lax', secure: true, httpOnly: true }`.

URL Pattern Use Cases

Below are common URL patterns you can achieve by combining `prefix`, `search`, and the corresponding detection config overrides. The available placeholders are `:siteId` and `:localeId`, which are resolved from the current site and locale, after alias mapping.

Site and Locale in the Path (Default)

This configuration is best for most multisite storefronts. It uses clean, fully deterministic URLs. It establishes a routing convention that prepends site and locale identifiers to subpage URLs while excluding specific utility paths. Because the defined prefix aligns with the framework's default patterns, the system can automatically detect and map these parameters without requiring additional manual overrides.

```
// config.server.ts
export default defineConfig({
  app: {
    url: {
      prefix: "[:siteId]:localeId",
      excludeRoutes: ["/resource/**", "/a
    ],
  },
  // No detection config override needed
});
```

This is the default configuration. Both site and locale are visible in every subpage URL. The default detection config expects site at path index 0 and



Homepage	/
Product (RefArchGlobal, en-GB)	/global/en-GB/product/123
Product (RefArch, en-US)	/us/en-US/product/123
Category (RefArchGlobal, it-IT)	/global/it-IT/category/womens

Locale Only in the Path

This configuration is best for single-site storefronts with multiple locales, or when the site is determined by cookie or domain. It defines a simplified URL structure that uses a single locale identifier as a prefix while exempting specific resource and action routes. Because the locale is now positioned at the very beginning of the URL path (index 0), a manual `localeDetectionConfig` override is included to ensure the system correctly identifies and parses the language setting from the updated path.

```
1 // config.server.ts
2 export default defineConfig({
3   app: {
4     url: {
5       prefix: "[:localeId]",
6       excludeRoutes: ["/resource/**", "/a
7     ],
8     // Required: locale moved from path in
9     localeDetectionConfig: { lookupFromPa
10   },
11 });
```

With this pattern, the site is resolved via the detection fallback chain – path detection doesn't find a valid site ID at index 0 (it sees a locale like `en-GB`), so it falls through to querystring, cookie, and header in that order.

You **must** update `localeDetectionConfig` to set `lookupFromPathIndex: 0`, because the locale is now the first path segment instead of the second.

Page	URL
Homepage	/
Product (en-GB)	/en-GB/product/123



Category (it-IT) /it-IT/category/womens

Site in the Path, Locale in Query Params

This configuration is best for: Shorter path segments while keeping locale in the URL for shareability. It establishes a hybrid URL structure that prepends a site identifier to the path while passing the locale as a query parameter. By defining `?lng=:localeId`, the application intelligently falls back to the search string when a valid locale isn't found in the secondary path segment. Since the site identifier remains at the primary position (index 0), the framework's default detection logic remains intact without requiring manual overrides.

```
1 // config.server.ts
2 export default defineConfig({
3   app: {
4     url: {
5       prefix: "[:siteId]",
6       search: "?lng=:localeId",
7       excludeRoutes: ["/resource/**", "/a
8     },
9     // No detection config override requi
10    // Site is still at path index 0 (defa
11    // Locale detection tries path index
12    // locale, and falls through to query
13  },
14 });
```

No detection config override is required. The site is still resolved from path index 0. For locale, the default detection tries path index 1 first, doesn't find a valid locale ID there, and falls through to the `?lng=` query parameter.

Page	URL
Homepage	/
Product (RefArchGlobal, en-GB)	/global/product/123? lng=en-GB
Product (RefArch, en-US)	/us/product/123?lng=en- US
Category (RefArchGlobal, it-IT)	/global/category/womens? lng=it-IT



instead managing both site and locale through query parameters. By defining the search pattern with `?site=:siteId&lng=:localeId`, the application allows standard path segments (like product names or IDs) to remain clean. Because the framework automatically falls back to the query string when it fails to find valid identifiers in the URL path, no manual detection overrides are necessary.

```
1 // config.server.ts
2 export default defineConfig({
3   app: {
4     url: {
5       search: "?site=:siteId&lng=:localeId",
6       excludeRoutes: ["/resource/**", "/api"],
7     },
8     // No detection config override required
9     // Path detection reads page path segments
10    // valid site/locale IDs, and falls through to the
11    // ?site=global&lng=en-GB.
12  },
13 });
```

Without a `prefix`, React Router doesn't need site/locale route params in its route definitions. No detection config override is required – the default detection tries path segments first, doesn't find valid site or locale IDs, and naturally falls through to the query parameters.

Page	URL
Homepage	/
Product	/product/123? site=global&lng=en-GB
Category	/category/womens? site=us&lng=en-US

Locale Only in Query Params

This configuration is best for single-site storefronts with locale-aware URLs without path changes. It focuses exclusively on a query-based locale identifier while removing site information from the URL entirely. By defining `?lng=:localeId`, the application allows the URL path to remain clean and content-focused, falling back to the search string to resolve the language. Since the site is no longer present in the URL, the framework automatically resolves it through alternative signals like cookies or headers, maintaining a streamlined configuration without the need for manual detection overrides.



```
4   url: {
5     search: "?lng=:localeId",
6     excludeRoutes: ["/resource/**", "/a
7   },
8   // No detection config override requi
9   // Path detection doesn't find valid
10  // and falls through. Site resolves f
11 },
12 };
```

No detection config override is required. The default detection falls through path segments to find the locale in the `?lng=` query parameter. Site is resolved from cookie or header via the same fallback chain.

Page	URL
Homepage	/
Product	/product/123?lng=en-GB
Category	/category/womens?lng=it-IT

No Prefix (Single Site)

This configuration is best for single-site, single-locale storefronts that don't need site or locale identifiers in the URL at all. It removes all identifiers from both the path and query string, producing the cleanest possible URLs. The multisite middleware still runs, but it resolves site and locale entirely from cookies and headers using the detection fallback chain. On a first visit with no cookies, the `defaultSiteId` and `defaultLocale` values from `config.server.ts` determine which site and locale are used.

```
</> |  | 
1 // config.server.ts
2 export default defineConfig({
3   app: {
4     defaultSiteId: "RefArch",
5     commerce: {
6       sites: [
7         {
8           id: "RefArch",
9           defaultLocale: "en-US",
10          defaultCurrency: "USD",
11          supportedLocales: [{ id: "en-US",
12            supportedCurrencies: ["USD"],
13          },
14        ],
15      },
16    url: {
17      prefix: "/",
18      excludeRoutes: ["/resource/**", "/a
19    },
```



No detection config override is required. The default detection tries path segments first, doesn't find valid site or locale IDs, and falls through to cookies and headers. Because there is only one site and one locale configured, the defaults are always used on the first visit and persisted via cookies for subsequent requests.

Page	URL
Homepage	/
Product	/product/123
Category	/category/womens

Access Site Context in Components

Use the `useSite()` hook to access the current site, locale, language, and currency in any React component:

```
1 import { useSite } from "@salesforce/sto
2
3 function MyComponent() {
4   const { site, locale, language, currency
5   // site: the resolved Site object (id, s
6   // locale: the full locale object from
7   // language: the i18next language string
8   // currency: the active currency code (c
9 }
```

`useSite()` throws if called outside of a `SiteProvider`. In the template, `SiteProvider` is mounted in `root.tsx` and wraps the entire app—no additional setup is needed.



containing `id`, an optional `default`, and an optional `preferredCurrency` field. `language` is the i18next language string used for translations. In most cases these have the same locale ID value, but they serve different purposes—use `locale` when you need the locale metadata, and `language` when you need the i18next language key.

Important Considerations

1. Default homepage behavior is cookie-driven.

By default, the template's `Link` component and `buildUrlFromContext` helper skip URL prefixing for `/`, so the homepage resolves site and locale from cookies. On a first visit with no cookie, the `defaultSiteId` determines which site is rendered. You can modify this behavior by updating `src/components/link/index.tsx` and `src/lib/url.server.ts`.

2. URL prefix changes require a rebuild.

`url.prefix` and `url.excludeRoutes` are protected paths. They can't be changed via `PUBLIC_` environment variables. Update `config.server.ts` and rebuild the application.

3. Detection config overrides are only required when path indexes shift.

The detection middleware uses a fallback chain—if it doesn't find a valid site or locale at the current source, it moves to the next one. In most non-default URL patterns, path detection simply fails to match and falls through to querystring or cookie. The only case where an override is **required** is when a path segment's position changes (e.g., locale moves from index 1 to index 0 in a locale-only prefix).

4. Query param keys are not arbitrary.

The locale query param key must be `lng` (matching the i18next cookie). The site key defaults to `site`. Custom keys require matching updates to the detection config.

5. Use multisite-aware navigation.

Always use `Link` and `NavLink` from `@/components/link`, `useNavigate` from `@/hooks/use-navigate`, and `buildUrlFromContext` for server-side redirects. These automatically apply the URL prefix. Using React Router's `Link` or `redirect` directly produces URLs without the prefix, resulting in 404 errors.

[Try for free](#)

unsupported locales or hide valid ones.

7. **Always include framework routes in `excludeRoutes`.** The `/resource/**` and `/action/**` patterns must remain in `excludeRoutes`. Removing them causes resource routes and server actions to be incorrectly prefixed with site/locale segments.

See Also

- [Project Configuration](#)
- [Internationalization](#)

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

DEVELOPER CENTERS

[Heroku](#)
[MuleSoft](#)
[Tableau](#)
[Commerce Cloud](#)
[Lightning Design System](#)
[Einstein](#)
[Quip](#)

POPULAR RESOURCES

[Documentation](#)
[Component Library](#)
[APIs](#)
[Trailhead](#)
[Sample Apps](#)
[AppExchange](#)

COMMUNITY

[Trailblazer Community](#)
[Events and Calendar](#)
[Partner Community](#)
[Blog](#)
[Salesforce Admins](#)
[Salesforce Architects](#)

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#) [Your Privacy Choices](#)

